

Basic Information

Student name: Yaotian Liu

Project title: RTLRepair-Agent: Verification-Guided Agentic RTL Generation

Repository / notebook link: <https://github.com/ytliu74/rtlrepair-agent>

Configuration location: README.md, .env.example, and examples/.

Section 1. Problem Definition

Research question: Can compiler and simulator feedback improve RTL generation over a one-shot LLM baseline?

- **User/task:** Help RTL designers turn a natural-language specification and fixed interface into synthesizable SystemVerilog.
- **Evaluation:** An independent testbench checks the generated RTL.
- **Pass:** Compilation and simulation exit zero; output includes `TEST_PASS` and no `TEST_FAIL`. Errors, timeouts, mismatches, or missing pass markers fail.

Section 2. Motivation and Project Scope

Generated RTL can contain syntax, timing, and functional errors. Open-source Icarus Verilog provides objective feedback for a generate–verify–repair workflow.

- **Implemented:** One-shot generation and automated verification on two designs.
- **Semester work:** Bounded repair and broader benchmark evaluation.
- **Stretch goal:** Synthesis/PPA feedback; physical design is outside the initial scope.

Section 3. Runnable Baseline

1. Python calls `gpt-4.1-2025-04-14` once per design using only the specification/interface; the testbench is withheld.
2. Save RTL, compile with `iverilog -g2012`, and simulate with `vvp`.
3. Report PASS/FAIL and save RTL, logs, timings, token usage, and JSON results.

Source: `src/{llm_client,rtl_generator,simulator,baseline}.py`. This one-shot control has **no repair or automatic retry**.

Section 4. Test Case and Baseline Output

- **8-bit counter:** Synchronous reset, enable/hold, and wraparound; **853 checks passed**.
- **16×8 FIFO:** Registered reads, ordering, occupancy flags, and pointer wrap. Simultaneous read/write accepts both at full and only the write at empty (no bypass). Independent queue scoreboard: **2,233 checks / 744 cycles passed**.

On September 6, 2026, both designs compiled, simulated, and returned **TEST_PASS**, including fresh-clone reruns. First-run generation: counter 1.985 s / 329 tokens; FIFO 3.478 s / 1,099 tokens. The FIFO had a nonfatal `ignored-unique case` warning. The genuine terminal screenshot below records the initial suite.

Section 5. Reproducibility and Run Instructions

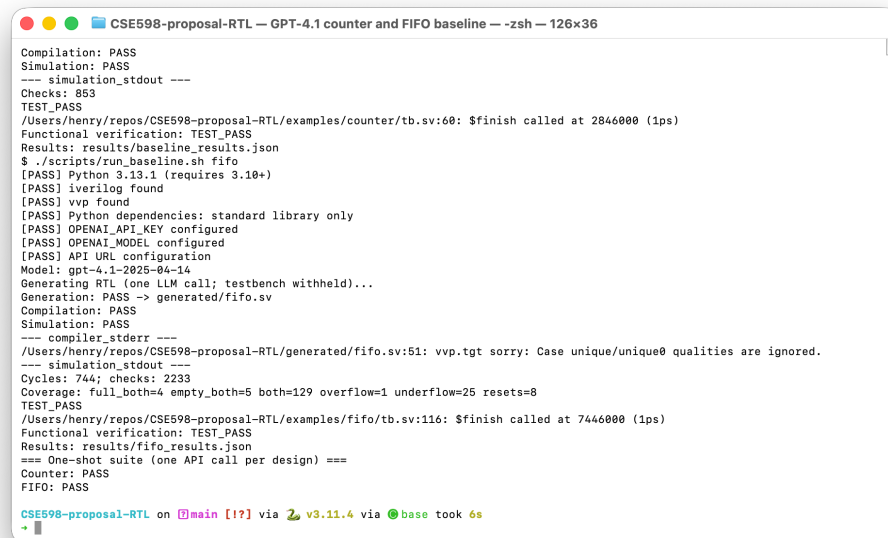
1. Clone the linked repository and enter `rtlrepair-agent`. Install Python 3.10+ and Icarus Verilog (macOS: `brew install icarus-verilog`).
2. Run `python3 -m venv .venv`, then source `.venv/bin/activate` and `python -m pip install -r requirements.txt` (no third-party Python packages).
3. Copy `.env.example` to `.env`; set `OPENAI_API_KEY` and keep `OPENAI_MODEL=gpt-4.1-2025-04-14`. Model access is required.
4. Run `./scripts/run_examples.sh` (one call per design; two total).
 - **Inputs:** `examples/{counter,fifo}/spec.txt` and `tb.sv`.
 - **Outputs:** `generated/{counter,fifo}.sv`, `results/{baseline,fifo}_results.json`, and `artifacts/{baseline,fifo}_output.txt`.
 - **Setup details:** README includes Linux instructions and offline tests; **20 tests pass**.

Section 6. Initial Evaluation Plan

- **Comparison:** One-shot vs. bounded repair, with the same model, initial prompt, tasks, and test-benches over repeated trials. Include budget-matched resampling; retain failures.
- **Metrics:** Functional pass rate (primary); compile rate, iterations, calls, tokens, latency, and measurable cost (secondary).
- **Expansion:** More curated tasks, then consider VerilogEval or RTLLM.

Section 7. Limitations and Next Steps

- **Limits:** Two tasks do not establish generality or model ranking. Simulation checks only tested behavior, not synthesizability. Results depend on test coverage, model variability, and API availability.
- **Next:** Implement diagnosis and bounded repair, then expand evaluation.



```
CSE598-proposal-RTL — GPT-4.1 counter and FIFO baseline — -zsh — 126x36

Compilation: PASS
Simulation: PASS
--- simulation_stdout ---
Checks: 853
TEST_PASS
/Users/henry/repos/CSE598-proposal-RTL/examples/counter/tb.sv:60: $finish called at 2846000 (1ps)
Functional verification: TEST_PASS
Results: results/baseline_results.json
$ ./scripts/run_baseline.sh fifo
[PASS] Python 3.13.1 (requires 3.10+)
[PASS] iverilog found
[PASS] vvp found
[PASS] Python dependencies: standard library only
[PASS] OPENAI_API_KEY configured
[PASS] OPENAI_MODEL configured
[PASS] API URL configuration
Model: gpt-4.1-2025-04-14
Generating RTL (one LLM call; testbench withheld)...
Generation: PASS -> generated/fifo.sv
Compilation: PASS
Simulation: PASS
--- compiler_stderr ---
/Users/henry/repos/CSE598-proposal-RTL/generated/fifo.sv:51: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
--- simulation_stdout ---
Cycles: 744; checks: 2233
Coverage: full_both=4 empty_both=5 both=129 overflow=1 underflow=25 resets=8
TEST_PASS
/Users/henry/repos/CSE598-proposal-RTL/examples/fifo/tb.sv:116: $finish called at 7446000 (1ps)
Functional verification: TEST_PASS
Results: results/fifo_results.json
=== One-shot suite (one API call per design) ===
Counter: PASS
FIFO: PASS

CSE598-proposal-RTL on [main [!]] via v3.11.4 via base took 6s
```

Figure 1: Actual GPT-4.1 run: FIFO verification and counter/FIFO PASS summary.